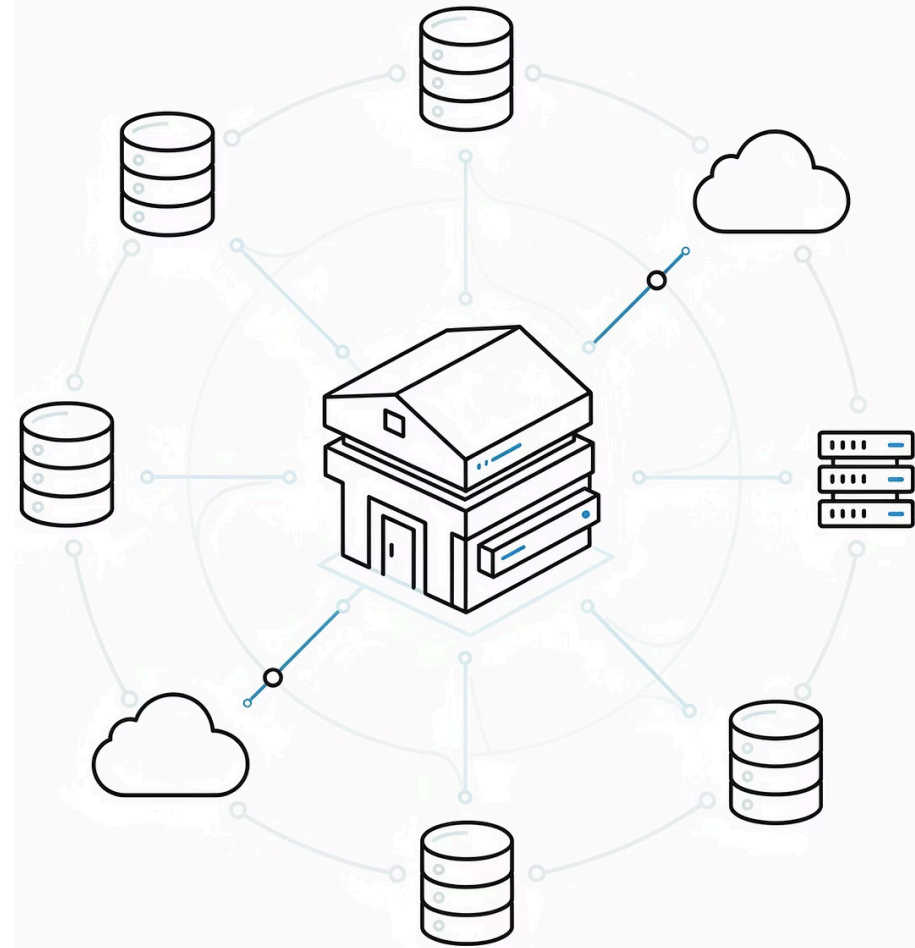


AULA 7 – BI TURMA UN

UNICEUB

Evoluindo o Data Warehouse com PostgreSQL



Objetivos desta Aula

01

Criar um novo banco de dados no PostgreSQL, separando o ambiente analítico do operacional e iniciando a estrutura do Data Warehouse.

03

Importar o arquivo CSV bruto para o schema repositorio, preservando os dados originais como fonte de construção do DW.

05

Criar e popular a dim_cliente, estruturando os dados de clientes para facilitar análises futuras.

07

Criar a tabela fato_vendas no schema dw, definindo métricas e relacionamentos com as dimensões.

09

Validar o modelo estrela criado, garantindo consistência, integridade e qualidade dos dados.

02

Criar os schemas repositorio e dw, organizando o projeto entre dados brutos e modelo analítico.

04

Criar e popular a dim_produto, extraíndo atributos dos produtos e eliminando duplicidades com SELECT DISTINCT.

06

Criar e popular a dim_filial, organizando as unidades de negócio dentro do modelo estrela.

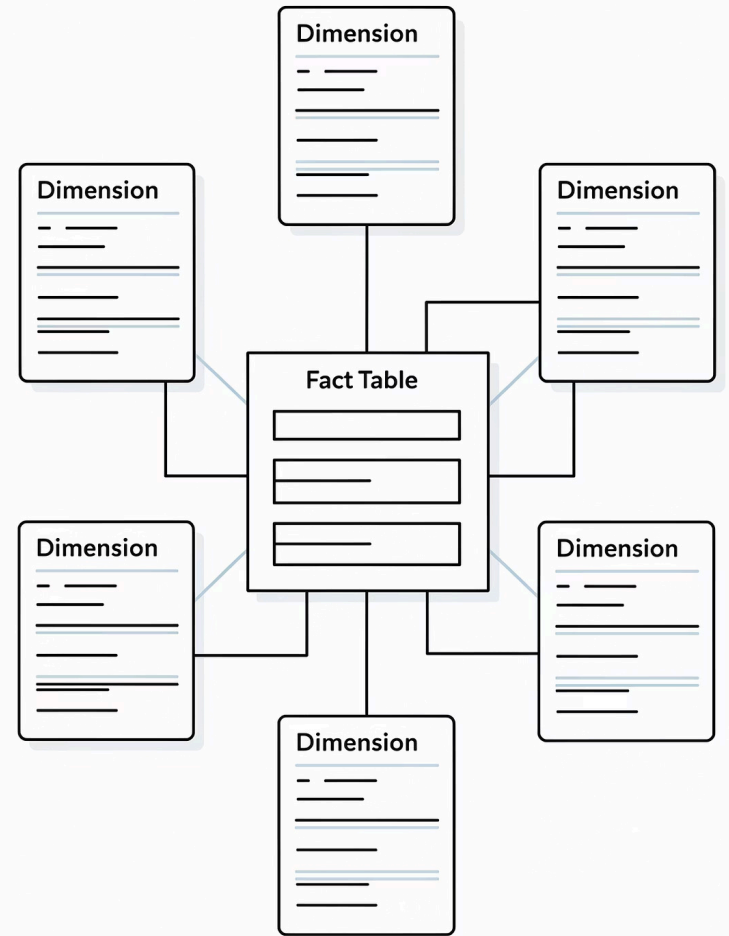
08

Popular a fato_vendas utilizando JOINS, transformando dados brutos em um modelo analítico estruturado.

10

Deixar o Data Warehouse pronto para consumo, habilitando a criação de dashboards e análises profissionais.

Construindo o Modelo Estrela no PostgreSQL



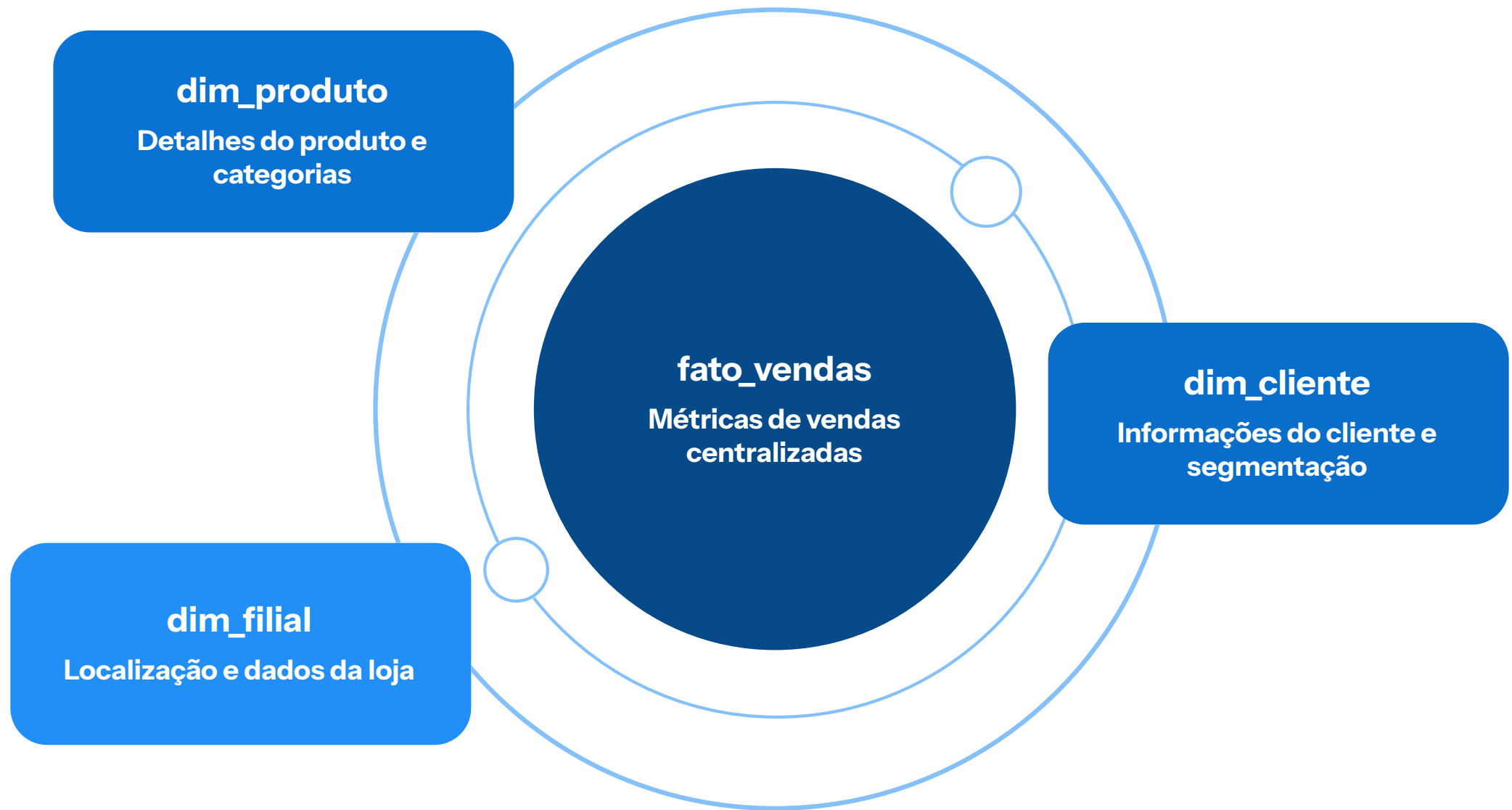
Objetivo desta etapa

Criar no schema `dw` as tabelas:

- `dim_produto`
- `dim_cliente`
- `dim_filial`
- `fato_vendas`

Essas tabelas serão alimentadas a partir da tabela bruta que está no schema `repositorio`.

Estrutura do Modelo Estrela



A tabela `fato_vendas` será o coração do nosso modelo, centralizando as métricas e sendo ligada diretamente às tabelas de dimensão: `dim_produto`, `dim_cliente` e `dim_filial`. Essa estrutura otimiza consultas e facilita análises.

Primeiros Passos: Configurando o Ambiente

Vamos iniciar a preparação do nosso ambiente no PostgreSQL para receber e organizar os dados brutos antes da transformação para o modelo estrela.

1

Criar o Schema repositorio

Um espaço dedicado no banco de dados para armazenar os dados brutos, mantendo-os isolados e organizados para processamentos futuros.

```
CREATE SCHEMA IF NOT EXISTS repositorio;
```

2

Importar o Dataset

Supermercado_ABC_100k

Carregar o arquivo CSV fornecido para uma nova tabela dentro do schema repositorio. Esta tabela será a base de onde extrairemos as dimensões e fatos.

Dataset: Supermercado_ABC_100k

Tempo estimado para esta etapa: **20 minutos**

Criando a Tabela Vendas no Schema repositorio

Esta etapa define a estrutura da tabela que receberá os dados brutos do dataset Supermercado_ABC_100k. É crucial que os tipos de dados correspondam aos do arquivo CSV para uma importação sem erros, preparando o ambiente para as próximas fases de transformação.

```
CREATE TABLE repositorio.vendas (  
  id_item TEXT,  
  id_venda TEXT,  
  data_hora TEXT,  
  filial_id TEXT,  
  filial TEXT,  
  cidade TEXT,  
  estado TEXT,  
  caixa TEXT,  
  operador_caixa TEXT,  
  id_cliente TEXT,  
  nome_cliente TEXT,  
  cidade_cliente TEXT,  
  estado_cliente TEXT,  
  segmento_cliente TEXT,  
  produto TEXT,  
  categoria TEXT,  
  subcategoria TEXT,  
  marca TEXT,  
  quantidade TEXT,  
  preco_unitario TEXT,  
  desconto_valor TEXT,  
  valor_liquido TEXT,  
  custo_total TEXT,  
  imposto TEXT,  
  comissao TEXT,  
  forma_pagamento TEXT  
)
```

Script para carregar o CSV

Se o arquivo estiver no servidor/local acessível ao PostgreSQL:

```
COPY dw.ecommerce_100k  
FROM 'C:\Ecommerce_100k.csv'  
DELIMITER ';'   
CSV HEADER  
ENCODING 'UTF8';
```


Póximo passo: Criar o Schema DW

Para organizar o nosso ambiente e preparar o terreno para o Data Warehouse, vamos criar um schema específico. Este será o local dedicado onde residirão todas as nossas tabelas dimensionais e de fatos.

```
CREATE SCHEMA IF NOT EXISTS dw;
```

A criação do schema dw é fundamental para manter os dados do Data Warehouse isolados e estruturados, facilitando a Gestão e o consumo por ferramentas de BI como o Power BI.

Criando a dimensão produto

A tabela `dim_produto` é crucial para analisar as vendas sob a perspectiva dos produtos. Ela armazenará atributos descritivos essenciais, permitindo análises aprofundadas sobre o desempenho de cada item no nosso catálogo.

- `produto`: O nome específico do item ou serviço.
- `categoria`: O grupo principal ao qual o produto pertence (ex: Eletrônicos, Alimentos).
- `subcategoria`: Um subgrupo mais detalhado dentro da categoria (ex: Smartphones, Laticínios).

Todos esses dados serão extraídos da tabela bruta `repositorio.Vendas`, garantindo a integridade e rastreabilidade da informação.

Script para Criar a Tabela `dim_produto`

```
CREATE TABLE dw.dim_produto (  
  id_produto SERIAL PRIMARY KEY,  
  produto VARCHAR(150),  
  categoria VARCHAR(100),  
  subcategoria VARCHAR(100)  
);
```

Populando a dim_produto

Com a estrutura da tabela `dw.dim_produto` já definida, o próximo passo crucial é preenchê-la com os dados exclusivos de produtos da nossa fonte bruta. Isso garante que cada item, categoria e subcategoria seja armazenado uma única vez na dimensão, otimizando as consultas futuras.

Script para Inserir Dados na dim_produto

```
INSERT INTO dw.dim_produto (produto, categoria, subcategoria)
SELECT DISTINCT
    produto,
    categoria,
    subcategoria
FROM repositorio.Vendas
WHERE produto IS NOT NULL;
```

O que este comando realiza?

→ Lê a Tabela Bruta

A consulta extrai informações diretamente da tabela `repositorio.Vendas`, que contém todos os dados transacionais originais.

→ Identifica Combinações Únicas

A cláusula `DISTINCT` é aplicada para garantir que apenas as combinações únicas de produto, categoria e subcategoria sejam selecionadas, evitando redundância.

→ Evita Duplicidade na Dimensão

Ao inserir apenas valores distintos, a `dim_produto` é populada sem registros repetidos, mantendo a integridade e a eficiência do nosso modelo dimensional.

Criando a Dimensão Cliente

A dimensão `dim_cliente` é essencial para aprofundar a análise do comportamento de compra, vinculando cada venda a informações demográficas importantes do cliente. Ela nos permitirá segmentar e entender melhor nosso público.

Esta dimensão conterá os seguintes atributos:

- `id_cliente_dw`: Chave primária artificial para o Data Warehouse.
- `id_cliente`: O identificador original do cliente na fonte de dados.
- `cidade_cliente`: A cidade de residência do cliente.
- `estado_cliente`: O estado de residência do cliente.

Script para Criar a Tabela `dim_cliente`

```
CREATE TABLE dw.dim_cliente (  
  id_cliente_dw SERIAL PRIMARY KEY,  
  id_cliente INT,  
  cidade_cliente VARCHAR(100),  
  estado_cliente VARCHAR(10)  
);
```

Populando a dim_cliente

Com a estrutura da dimensão dim_cliente pronta, o próximo passo é preenchê-la com dados únicos dos clientes da nossa fonte bruta. Isso garante que cada cliente seja registrado uma única vez, promovendo a consistência e a eficiência para análises futuras.

Script para Inserir Dados na dim_cliente

```
INSERT INTO dw.dim_cliente (id_cliente, cidade_cliente, estado_cliente)
SELECT DISTINCT
  CAST(id_cliente AS INTEGER),
  cidade_cliente,
  estado_cliente
FROM repositorio.Vendas
WHERE id_cliente IS NOT NULL;
```

O que este comando realiza?

→ Lê a Tabela Bruta

A consulta extrai informações diretamente da tabela `repositorio.Vendas`, que contém todos os dados transacionais originais.

→ Identifica Clientes Únicos

A cláusula `DISTINCT` é utilizada para selecionar apenas identificadores de cliente, cidade e estado únicos, prevenindo a duplicação de registros.

→ Garante Unicidade na Dimensão

Ao inserir somente valores distintos, a `dim_cliente` é populada sem repetições, assegurando a integridade e otimizando a performance do nosso modelo dimensional.

Criando a Dimensão Filial

A dimensão `dim_filial` é essencial para analisar as vendas sob a perspectiva da localização física. Ela armazena os dados das unidades de negócio onde as transações ocorrem, permitindo análises regionais e de desempenho por ponto de venda.

Ela deve conter os seguintes atributos para cada filial:

- **Filial:** O nome ou identificador da unidade de negócio.
- **Cidade:** A cidade onde a filial está localizada.
- **Estado:** O estado onde a filial opera.

Script para Criar a Tabela `dim_filial`

```
CREATE TABLE dw.dim_filial (  
  id_filial SERIAL PRIMARY KEY,  
  filial VARCHAR(100),  
  cidade VARCHAR(100),  
  estado VARCHAR(10)  
);
```

Populando a dim_filial

Após criar a estrutura da tabela `dw.dim_filial`, o passo seguinte é preenchê-la com os dados únicos das filiais extraídos da nossa fonte transacional. Isso garante que cada unidade de negócio seja representada uma única vez na dimensão, otimizando as consultas e análises por localização.

Script para Inserir Dados na `dim_filial`

```
INSERT INTO dw.dim_filial (filial, cidade, estado)
SELECT DISTINCT
    filial,
    cidade,
    estado
FROM repositorio.Vendas
WHERE filial IS NOT NULL;
```

O que este comando realiza?

→ Lê a Tabela Bruta

A consulta busca informações diretamente da tabela `repositorio.Vendas`, que contém os registros originais das transações de venda, incluindo dados das filiais.

→ Identifica Filiais Únicas

A cláusula `DISTINCT` é aplicada para selecionar apenas as combinações únicas de filial, cidade e estado, evitando a inserção de registros duplicados.

→ Garante Unicidade na Dimensão

Ao inserir somente valores distintos, a `dim_filial` é populada sem repetições, assegurando a integridade dos dados e a eficiência do nosso modelo dimensional para análises geográficas.

Criando a Tabela fato_vendas

A tabela de fatos é o coração do nosso modelo estrela, armazenando as métricas e os links para as dimensões. Ela centraliza as informações transacionais e quantificáveis do nosso negócio.

O que a fato_vendas deve conter?

1

Chaves de Ligação

- id_produto
- id_cliente_dw
- id_filial

2

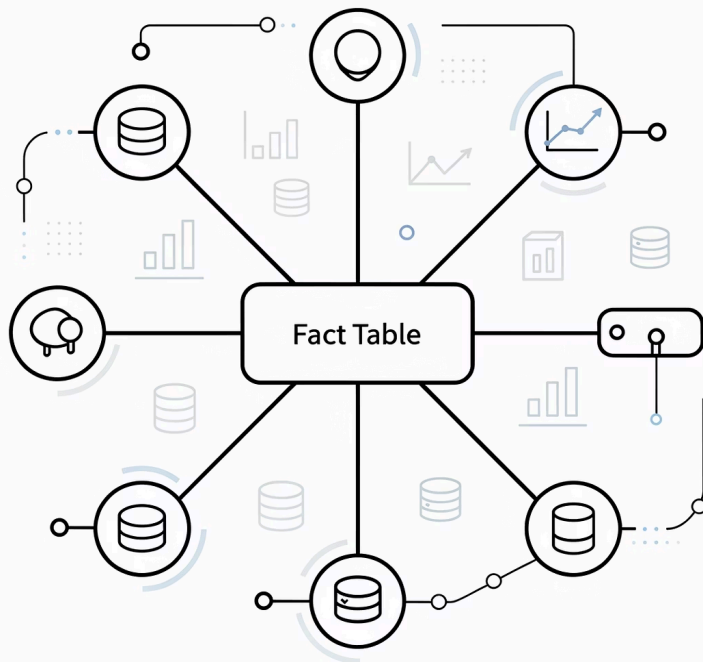
Métricas

- quantidade
- valor_total
- custo_total

3

Informações do Evento

- id_venda
- data



Criando a Tabela fato_vendas

A tabela fato_vendas é o elemento central do nosso esquema estrela. Ela armazenará as métricas quantificáveis de cada transação de venda e atuará como a ponte para as nossas dimensões recém-criadas. A seguir, o script para a sua criação no PostgreSQL:

```
CREATE TABLE dw.fato_vendas (  
  id_item INT PRIMARY KEY,  
  id_venda INT,  
  data TIMESTAMP,  
  id_produto INT,  
  id_cliente_dw INT,  
  id_filial INT,  
  quantidade INT,  
  valor_total NUMERIC(12,2),  
  custo_total NUMERIC(12,2),  
  FOREIGN KEY (id_produto) REFERENCES dw.dim_produto(id_produto),  
  FOREIGN KEY (id_cliente_dw) REFERENCES dw.dim_cliente(id_cliente_dw),  
  FOREIGN KEY (id_filial) REFERENCES dw.dim_filial(id_filial)  
);
```

Pontos Chave da Estrutura

- A PRIMARY KEY composta por (id_venda, id_produto) garante a unicidade de cada linha, permitindo registrar múltiplos produtos em uma única venda.
- As FOREIGN KEYS (id_produto, id_cliente_dw, id_filial) conectam esta tabela de fatos às suas respectivas tabelas de dimensão, estabelecendo as relações necessárias para a análise.
- Colunas como quantidade, valor_total e custo_total armazenam as métricas numéricas essenciais para os nossos relatórios de BI.
- A coluna data do tipo TIMESTAMP registra o momento exato da transação, crucial para análises temporais.

Carregando a fato_vendas

Com as tabelas de dimensão criadas e a estrutura da fato_vendas definida, o próximo passo crucial é popular a tabela de fatos com os dados transacionais brutos, fazendo a ponte para as dimensões corretas. Isso garante que cada transação de venda esteja ligada ao seu produto, cliente e filial correspondentes no Data Warehouse.

O desafio é **como alimentar a fato?** Precisamos converter os identificadores originais presentes na tabela de vendas bruta nos IDs das dimensões recém-criadas. Este processo envolve buscar os IDs internos de cada dimensão para cada registro de venda.

01

Mapeamento de Produto

O identificador de produto da tabela bruta (id_produto ou nome do produto) precisa ser ligado ao id_produto da dimensão dw.dim_produto.

02

Mapeamento de Cliente

O identificador de cliente da tabela bruta (id_cliente) precisa ser ligado ao id_cliente_dw da dimensão dw.dim_cliente.

03

Mapeamento de Filial

O identificador de filial da tabela bruta (filial ou id_filial) precisa ser ligado ao id_filial da dimensão dw.dim_filial.

Inserindo Dados na fato_vendas: O Poder dos JOINS

Agora que todas as dimensões estão criadas e populadas, é hora de preencher a tabela de fatos. O script abaixo demonstra como integrar os dados transacionais brutos com as chaves das dimensões, garantindo a integridade e a capacidade analítica do nosso Data Warehouse.

```
INSERT INTO dw.fato_vendas (  
  id_item,  
  id_venda,  
  data,  
  id_produto,  
  id_cliente_dw,  
  id_filial,  
  quantidade,  
  valor_total,  
  custo_total  
)  
SELECT  
  b.id_item::INTEGER,  
  b.id_venda::INTEGER,  
  b.data_hora::TIMESTAMP,  
  p.id_produto,  
  c.id_cliente_dw,  
  f.id_filial,  
  b.quantidade::INTEGER,  
  REPLACE(b.valor_liquido, ',', '. '::NUMERIC(12,2),  
  REPLACE(b.custo_total, ',', '. '::NUMERIC(12,2)  
FROM repositorio.vendas b  
JOIN dw.dim_produto p  
  ON b.produto = p.produto  
  AND b.categoria = p.categoria  
  AND b.subcategoria = p.subcategoria  
JOIN dw.dim_cliente c  
  ON b.id_cliente::INTEGER = c.id_cliente  
  AND b.cidade_cliente = c.cidade_cliente  
  AND b.estado_cliente = c.estado_cliente  
JOIN dw.dim_filial f  
  ON b.filial = f.filial  
  AND b.cidade = f.cidade  
  AND b.estado = f.estado;
```

Como este Script Transforma os Dados?

→ **Seleção e Substituição de Chaves**

O comando `SELECT` busca os dados brutos e, crucialmente, substitui os identificadores originais pelas chaves primárias das tabelas de dimensão (`id_produto`, `id_cliente_dw`, `id_filial`).

→ **Conexão com Dimensões**

Múltiplos `JOINS` são utilizados para ligar a tabela bruta `repositorio.fato_vendas_bruta` às dimensões `dim_produto`, `dim_cliente` e `dim_filial`.

→ **Garantia de Modelagem Estrela**

As condições de `JOIN` em múltiplos atributos (e.g., `produto`, `categoria`, `subcategoria`) asseguram que a ligação seja precisa e que os dados sejam formatados corretamente para o modelo estrela.

Conferindo a Integridade do Modelo Estrela

Após carregar os dados na tabela `fato_vendas`, é fundamental realizar uma verificação rápida para garantir que todas as conexões com as tabelas de dimensão foram estabelecidas corretamente. Este passo assegura que o modelo estrela está íntegro e pronto para análises.

```
SELECT
  f.id_venda,
  p.produto,
  c.cidade_cliente,
  fil.filial,
  f.quantidade,
  f.valor_total
FROM dw.fato_vendas f
JOIN dw.dim_produto p
  ON f.id_produto = p.id_produto
JOIN dw.dim_cliente c
  ON f.id_cliente_dw = c.id_cliente_dw
JOIN dw.dim_filial fil
  ON f.id_filial = fil.id_filial
LIMIT 10;
```

Validação das Chaves Estrangeiras

Este script testa se as chaves estrangeiras na `fato_vendas` estão se conectando corretamente às chaves primárias das dimensões, trazendo atributos descritivos como nome do produto, cidade do cliente e filial.

Verificação de Dados

Ao exibir as primeiras 10 linhas (`LIMIT 10`), podemos realizar uma inspeção visual rápida para confirmar que os dados esperados das dimensões estão aparecendo ao lado dos fatos da venda, indicando que os JOINS funcionaram.

Preparação para Análise

A execução bem-sucedida desta consulta nos dá confiança de que nosso Data Warehouse está estruturado de forma consistente e que podemos prosseguir com as consultas analíticas mais complexas.